

Lecture - 9(Array problems)* Nested loop :

	0	1	2	3	4
arr :	2	3	13	8	7

print same array 5 times?

Approach 1 :

Repeat for loops 5 times?

Code :

```

for (int i = 0; i < n; i++) {
    cout << arr[i];
}
for (int i = 0; i < n; i++) {
    cout << arr[i];
}

```

But what if we have to print same array
100 times.

We use nested loop:

```

for (int i = 0; i < 5; i++) {
    for (int j = 0; j < n; j++) {
        cout << arr[j] << " ";
    }
    cout << endl;
}

```


problem 2:

arr : 2 3 13 8 7

output :

row	col	2	3	13	8	7
0	5					
1	4	2	3	13	8	
2	3	2	3	13		
3	2	2	3			
4	1	2				

$j < 5 - i$
 $col < 5 - row$

After each row the last col get eliminated.

code :

```

for (int row = 0; row < 5; row++) {
    for (int col = 0; col < 5 - row; col++) {
        cout << arr[col] << " ";
    }
    cout << endl;
}

```


Problem 3:

arr: 2 3 13 8 7

Output:

row	col					
0	5	2	3	13	8	7
1	4	3	13	8	7	
2	3	13	8	7		
3	2	8	7			
4	1	7				

gn 0th row → 0 element delete from first
(0-4) print

gn 1st row → 1 element
(1-4) print

gn 2nd row → 2 element
(2-4) print

gn 3rd row → 3 element
(3-4) print

gn 4th row → 4 element
(4-4) print

```

for (int i=0; i<5; i++) {
    for (int j=i; j<5; j++) {
        cout << arr[j] << " ";
    }
    cout << endl;
}

```


Problem 4:

array :		3	13	12	8	4
row	col					
0	(n-row)	4	8	12	13	3
1	(n-row)	8	12	13	3	
2	(n-row)	12	13	3		
3	(n-row)	13	3			
4	(n-row)	3				

For 1st row : 0 element deleted and print in reverse
(4 → 0)

For 2nd row : 1 element deleted from last and
print in reverse.
(3 → 0)

For 3rd row : 2 element deleted from last and
print in reverse.
(2 → 0)

for 4th row : 3 element deleted from last and
print in reverse
(1 → 0).

for 5th row : (0 → 0)

code :

```
for (int row = 0; row < 5; row++) {
    for (int col = 4 - row; col >= 0; col--) {
        cout << arr[col] << " ";
    }
    cout << endl;
}
```


Problem 5:

array : { 3 13 12 8 4 }

output :

row	col					
0	4-0	4	8	12	13	3
1	4-1	4	8	12	13	
2	4-2	4	8	12		
3	4-3	4	8			
4	4-4	4				

↳ all in reverse order:

Code :

```
for (int i = 0; i < 5; i++) {
    for (int j = 4; j >= i; j--) {
        cout << arr[j] << " ";
    }
    cout << endl;
}
```


problem 6: Find duplicate in an array

Example :

$N = 4$ input: 0 to $N-1$
 $arr[] = \{0, 3, 1, 2\}$ 0 to 3

output = -1

↳ because No any element is repeated.

Example :

$N = 5$
 $arr[] = \{2, 3, 1, 2, 3\}$

output = $\{2, 3\}$.

Input: 0 to $N-1$
 0 to 4

2 3 1 2 3

No.	Count
0	0
1	1
2	2
3	2
4	0

} return because they are repeated.

0	1	2	3	4
0	x_1	$x_1 x_2$	$x_1 x_3$	0

✓
 greater than 1
 print as output.

Code :

```
int arr[] = { 2, 3, 1, 2, 3 };
```

```
int n = 5;
```

```
int Count[n] = { 0 };
```

```
for (int i = 0; i < n; i++) {
```

```
    int index = arr[i];
```

```
    Count[index]++;
```

```
}
```

```
for (int i = 0; i < n; i++) {
```

```
    if (Count[i] > 1) {
```

```
        cout << i << " ";
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```


Sorting

arr : 2 7 11 4

The data is sorted when this is either in ascending or descending.

ascending : 2 4 7 11

descending : 11 7 4 2

Sort of array of 0's, 1's and 2's.

$N = 5$

arr[] = { 0, 1, 2, 0, 1 }

Output = { 0, 0, 1, 1, 2 }

count_zero = 0 / 2

Count_one = 0 / 2

Count_two = 0 / 1

Code :

```
for (int i = 0; i < Count_zero; i++) {
```

```
    arr[i] = 0;
```

```
}
```

```
for (int i = Count_zero; i < Count_zero + Count_one; i++) {
```

```
    arr[i] = 1;
```

```
}
```



```
for (int i = Count_zero + Count_one; i < n; i++) {
    arr[i] = 2;
}
```

(Lecture - 10)

(Sorting : Basic)

* Selection Sort :

arr : { 8 , 4 , 13 , 7 , 5 }

Output : 4 5 7 8 13

8 | 4 13 7 5
 ↓ min
 Choose minimum from this side

Swap this With 1st element

4 | 8 13 7 5
 ↑ minimum

choose min

Swap With 2nd element

4 5 | 13 7 8
 ↑ min

choose min

Swap With 3rd element

4 5 7 | 13 8

Teacher's Signature ↑ min

chose minimum
swap with 4th element.

4 5 7 8 13

→ final result.

code?

```
for (int i = 0; i < n; i++) {
    int index = i;
    for (int j = i; j < n-1; j++) {
        if (arr[index] > arr[j]) {
            index = j;
        }
    }
    int temp = arr[index];
    arr[index] = arr[i];
    arr[i] = temp;
}
```

*

Bubble sort

arr[]: { 7 2 4 3 6 5 }

In this we compare two value and swap it
In every pass one element goes at its right place.
The last element goes at right place

We have to do (n-1) pass to complete this.

pass 1:

(7) (2) 4 3 6 5

$7 > 2$ (swap)

2 (7) (4) 3 6 5

$7 > 4$ (swap)

2 4 (7) (3) 6 5

$7 > 3$ (swap)

2 4 3 (7) (6) 5

$7 > 6$ (swap)

2 4 3 6 (7) (5)

$7 > 5$ (swap)

2 4 3 6 5 | 7

→ got its right place.

pass 2:

(2) (4) 3 6 5 | 7

$2 < 4$ (No swap)

2 (4) (3) 6 5 | 7

$4 > 3$ (swap)

2 3 (4) (6) 5 | 7

$4 < 6$ (No swap)

2 3 4 (6) (5) | 7

$6 > 5$ (swap)

2 3 4 5 | 6 7

pass 3:

(2) (3) 4 5 | 6 7

$2 < 3$ (No swap)

2 (3) (4) 5 | 6 7

$3 < 4$ (No swap)

2 3 (4) (5) | 6 7

$4 < 5$ (No swap)

pass 4 :

(2) (3) 4 | 5 6 7

$2 < 3$ (No swap)

2 (3) (4) | 5 6 7

$3 < 4$ (No swap)

pass 5 :

(2) (3) | 4 5 6 7

$2 < 3$ (No swap)

2 3 4 5 6 7

↳ sorted

Code :

```
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < (n - 1) - i; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```


* Insertion Sort :

arr[] : 4 6 3 11 7 2

sorted

(1-0)

4 | 6 3 11 7 2
 ↑ place at correct place in left

4 6 | 3 11 7 2 (2-0)

3 4 6 | 11 7 2 (3-0)

3 4 6 11 | 7 2 (4-0)

3 4 6 7 11 | 2 (5-0)

2 3 4 6 7 11

Code : → sorted.

```

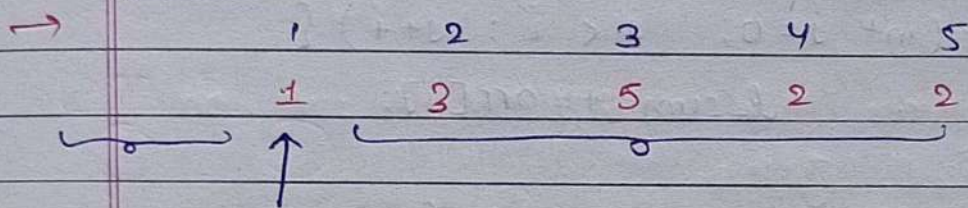
for (int i = 0 ; i < n-1 ; i++) {
    for (int j = i ; j >= 0 ; j--) {
        if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
        else
            break;
    }
}

```


* Problem : Equilibrium point

The left side subarray and right side subarray of an element is equal, then the element is known as Equilibrium point:

Example:

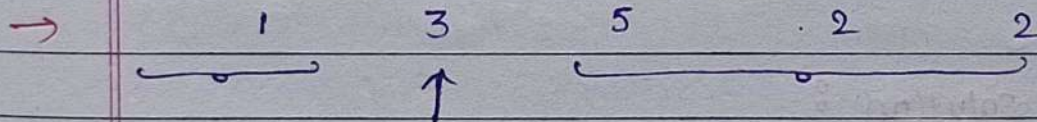


$$l_sum = 0$$

$$right = 3 + 5 + 2 + 2 = 12$$

$$l_sum \neq right$$

1 is not Equilibrium point

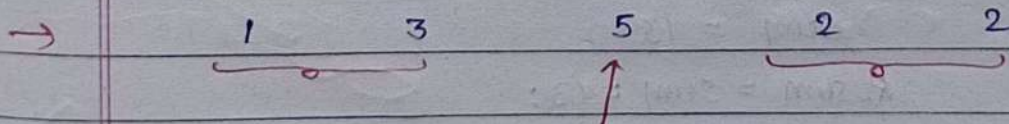


$$l_sum = 1$$

$$right = 5 + 2 + 2 = 9$$

$$l_sum \neq right$$

3 is not Equilibrium point



$$l_sum = 1 + 3 = 4$$

$$right = 2 + 2 = 4$$

$$l_sum = right$$

5 is the Equilibrium point.

Code: (Brute force) T.C: $O(N^2)$

```

for (int i = 0; i < n; i++) {
    int r_sum = 0;
    int l_sum = 0;
    for (int j = i + 1; j < n; j++) {
        r_sum += arr[j];
    }
    for (int j = 0; j < i; j++) {
        l_sum += arr[j];
    }
    if (l_sum == r_sum) {
        return i + 1;
    }
}
return -1;

```

Optimal solution:

1 3 5 2 2

calculate sum

sum = 13

R_sum = sum = 13;

→

1
↑

R_sum = 13 - 1 = 12
L_sum = 0

> Not Equal.

1 3 5 2 2
 ↑

$$R\text{-sum} = 12 - 3 = 9 \quad \text{> Not Equal}$$

$$L\text{-sum} = 1$$

1 3 5 2 2
 ↑

$$R\text{-sum} = 9 - 5 = 4 \quad \text{> Equal}$$

$$L\text{-sum} = 4$$

Equilibrium point.

Time complexity : $O(N)$

Code :

```

if (n == 1)
    return 1;

long totalSum = 0;
int res = -1;
long sum = 0;
for (int i = 0; i < n; i++)
    totalSum += arr[i];

for (int i = 1; i < n; i++) {
    sum += arr[i-1];

    if (totalSum - sum - arr[i] == sum) {
        res = i;
        break;
    }
}

return res == -1 ? -1 : res + 1;

```